MEET THE TOOLCHAIN

What actually turns your code into a program

WHY THIS CHAPTER

Before we write a single lesson in C++...

You need somewhere to write code, and something that turns it into a program you can run. This chapter is that "somewhere" - Windows, Linux, or Mac, whichever one you're on.

We won't write any new C++ in this chapter. Every lecture builds the exact same tiny program - the point is watching **how** it gets built, not what it does.

Four words you'll hear constantly

- **Compiler** - turns your `.cpp` source file into machine code the CPU can run (an *object file*).
- **Linker** - stitches your object file(s) together with any libraries they use, producing one runnable *executable*.
- **Build system** - automates calling the compiler and linker correctly, in the right order, with the right flags. In this course, that's **CMake**.
- **IDE** - the editor/UI wrapped around all of the above, so you don't run the compiler and linker by hand. In this course: **Visual Studio** or **Qt Creator**.

COMPILER + LINKER

Source code isn't a program yet

```
main.cpp  --[compiler]-->  main.o  --[linker]-->  rooster (or rooster.exe)
```

You get to the executable by running the compiler and linker and other tools on your source code.

WHERE CMAKE FITS

CMake doesn't compile anything itself

CMake reads a `CMakeLists.txt` and *generates* the files a build tool (like Ninja, or Visual Studio's own build engine) needs to actually call the compiler and linker, in the right order, with the right flags.

Every lecture folder in this course ships one:

```
project(rooster)
set(CMAKE_CXX_STANDARD 23)
add_executable(rooster main.cpp)
```

WHERE IDES FIT

Visual Studio and Qt Creator both call CMake

Neither IDE is its own separate build system in this course - both read the exact same `CMakeLists.txt`. What differs is *which compiler* each one hands that `CMakeLists.txt` to, and that's what the next two lectures are about.

THIS CHAPTER'S PLAN

Where we're headed

- **Windows:** Visual Studio, and Qt Creator with a Kit
- **Linux:** Qt Creator with a Kit
- **Mac:** Xcode, and Qt Creator with a Kit
- Checking whether your compiler is modern enough
- Docker, for when the compiler is **too old**
- CMake, the basics